TRAVAUX PRATIQUES

Terraform sur AWS

Des premières ressources aux modules réutilisables

TP guidé — fondamentaux Terraform

Objectif pédagogique
Construire progressivement une petite infrastructure AWS avec Terraform, comprendre les mécanismes essentiels du langage, puis structurer une partie de la configuration dans un module réutilisable.

Public : stagiaires ayant suivi les chapitres sur Terraform, les providers, les ressources, les outputs, les variables, les locals et les modules.

Environnement : AWS • Terraform CLI • terminal • éditeur de code

1. Présentation du TP

Vous allez construire une infrastructure web minimale dans AWS. Le serveur EC2 hébergera une page web Nginx accessible depuis Internet. Pour y parvenir, vous créerez un réseau virtuel, un sous-réseau public, une passerelle Internet, une table de routage et un groupe de sécurité.

Dans un second temps, vous rendrez la configuration paramétrable avec des variables et des locals, puis vous déplacerez la partie réseau dans un module local.

Important : le TP est volontairement limité aux notions étudiées. Il ne traite pas de RDS, Lambda, S3, IAM avancé, ni d'architecture de production.

Compétences visées

- Identifier le rôle du provider et des ressources Terraform.
- Initialiser un projet et comprendre le cycle init / plan / apply / destroy.
- Déclarer des ressources et exploiter les références entre elles.
- Exposer des informations avec les outputs.
- Paramétrer une configuration avec des variables typées et des validations.
- Factoriser des expressions avec locals.
- Créer, appeler et réutiliser un module local.

Architecture cible

Le schéma logique est le suivant :

Composant	Rôle
VPC	Réseau privé virtuel AWS
Subnet public	Sous-réseau dans lequel sera placée l'instance
Internet Gateway + route	Accès réseau vers Internet
Security Group	Autorise le trafic HTTP entrant sur le port 80
EC2	Instance Linux exécutant Nginx

Flux attendu : Internet → Internet Gateway → route du subnet public → Security Group → instance EC2 (HTTP/80).

2. Prérequis et règles de travail

- Un compte AWS de formation avec les droits nécessaires pour créer VPC, sous-réseau, Internet Gateway, routes, Security Groups et EC2.
- AWS CLI configuré avec des identifiants de formation. Vérifiez l'identité avec `aws sts get-caller-identity`.
- Terraform CLI installé. La configuration de ce TP utilise la contrainte `Terraform >= 1.5.0, < 2.0.0`.
- Provider AWS contraint à la famille `~> 6.0`. Le fichier de verrouillage généré par `terraform init` doit être conservé.
- Chaque stagiaire utilise un identifiant distinct, par exemple `stagiaire01`, afin d'éviter les collisions de noms.

Coûts : une instance EC2 et certaines ressources AWS peuvent entraîner une facturation. Utilisez exclusivement le compte de formation, la région et le type d'instance autorisés par le formateur. Détruisez l'infrastructure à la fin du TP. Ne partez pas du principe qu'une ressource est gratuite.

Convention de nommage

Dans ce TP, le préfixe de nommage sera construit à partir du nom du projet et de l'identifiant stagiaire. Exemple : `intro-terraform-stagiaire01`.

Structure finale visée

```
terraform-aws-fondamentaux/
■■■ versions.tf
■■■ provider.tf
■■■ variables.tf
■■■ locals.tf
■■■ main.tf
■■■ outputs.tf
■■■ terraform.tfvars
■■■ modules/
    ■■■ network/
        ■■■ main.tf
        ■■■ variables.tf
        ■■■ outputs.tf
```

3. Démarrage du projet

Étape 0 — Créer le répertoire de travail

Objectif : Disposer d'un espace de travail propre.

Créez le répertoire `terraform-aws-fondamentaux`, puis placez-vous dedans.

Créez les fichiers `versions.tf` et `provider.tf`.

Étape 1 — Déclarer les versions et le provider

Objectif : Indiquer à Terraform les versions compatibles et le provider à utiliser.

Dans `versions.tf`, déclarez les contraintes suivantes :

```
terraform {
  required_version = ">= 1.5.0, < 2.0.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}
```

Dans `provider.tf`, configurez le provider AWS. À ce stade, la région est volontairement écrite en dur ; elle sera remplacée par une variable à l'étape suivante.

```
provider "aws" {
  region = "eu-west-1"
}
```

Adaptez la région si le formateur vous en a attribué une autre.

Étape 2 — Initialiser le projet

Objectif : Télécharger le provider et préparer le répertoire de travail Terraform.

Exécutez les commandes suivantes :

```
terraform init
terraform version
terraform providers
```

Observez la création du répertoire `.terraform/` et du fichier `.terraform.lock.hcl`. Le fichier de verrouillage fixe les versions exactes sélectionnées pour les providers.

À retenir : `terraform init` prépare le projet ; il ne crée pas encore l'infrastructure AWS.

4. Première ressource et cycle Terraform

Étape 3 — Créer un VPC

Objectif : Déclarer une première ressource AWS et comprendre la relation entre type de ressource et nom local.

Dans `main.tf`, déclarez un VPC :

```
resource "aws_vpc" "formation" {
  cidr_block           = "10.20.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "intro-terraform-vpc"
  }
}
```

Le premier identifiant, `aws_vpc`, est le type de ressource fourni par le provider AWS. Le second, `formation`, est le nom local utilisé dans la configuration Terraform.

Étape 4 — Vérifier avant de créer

Objectif : Lire les contrôles statiques et le plan proposé par Terraform.

Exécutez successivement :

```
terraform fmt
terraform validate
terraform plan
```

Lisez le plan : repérez la ressource qui sera créée et les attributs qui seront transmis à AWS. À ce stade, aucun changement n'est encore appliqué.

Étape 5 — Créer la ressource

Objectif : Appliquer le plan et observer le résultat.

Exécutez `terraform apply`, lisez le plan puis confirmez avec `yes`.

Dans la console AWS, retrouvez le VPC créé. Comparez son identifiant AWS avec les informations affichées dans le terminal.

Terraform conserve les informations de la ressource dans son état, le fichier `terraform.tfstate`. Ne modifiez pas ce fichier manuellement.

5. Construire le réseau et l'instance

Vous allez compléter progressivement le réseau. Les références entre ressources permettront à Terraform de déterminer l'ordre de création.

Étape 6 — Créer l'Internet Gateway

Objectif : Relier le VPC à une passerelle Internet.

Ajoutez dans `main.tf` :

```
resource "aws_internet_gateway" "formation" {
  vpc_id = aws_vpc.formation.id

  tags = {
    Name = "intro-terraform-igw"
  }
}
```

La référence `aws_vpc.formation.id` crée une dépendance implicite : Terraform sait que le VPC doit exister avant de rattacher la passerelle.

Étape 7 — Créer un sous-réseau public

Objectif : Définir un sous-réseau dans le VPC.

Ajoutez :

```
resource "aws_subnet" "public" {
  vpc_id                  = aws_vpc.formation.id
  cidr_block              = "10.20.1.0/24"
  map_public_ip_on_launch = true

  tags = {
    Name = "intro-terraform-public"
  }
}
```

Étape 8 — Créer la table de routage

Objectif : Définir une route par défaut vers Internet et associer la table au sous-réseau.

Ajoutez les deux ressources suivantes :

```
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.formation.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.formation.id
  }

  tags = {
    Name = "intro-terraform-public-rt"
  }
}

resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}
```

Repérez les références entre la table de routage, l'Internet Gateway et le sous-réseau. Terraform construit son graphe de dépendances à partir de ces références.

Étape 9 — Créer un groupe de sécurité HTTP

Objectif : Autoriser les connexions HTTP entrantes sur le port 80.

Ajoutez :

```
resource "aws_security_group" "web" {
  name        = "intro-terraform-web"
  description = "Autoriser HTTP"
  vpc_id      = aws_vpc.formation.id

  ingress {
    description = "HTTP depuis Internet"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "intro-terraform-web-sg"
  }
}
```

Dans cet exercice, HTTP est ouvert à Internet pour permettre le test du site. N'ajoutez pas de règle SSH ouverte à tous : le TP ne nécessite pas de connexion SSH.

Étape 10 — Récupérer une AMI Amazon Linux

Objectif : Utiliser une data source pour retrouver une image existante dans la région choisie.

Ajoutez dans `main.tf` :

```
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-2023.*-x86_64"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}
```

Une data source lit une information existante dans AWS ; elle ne crée pas l'AMI. Le filtre sélectionne une image Amazon Linux 2023 correspondant au motif demandé.

Étape 11 — Créer l'instance EC2

Objectif : Déployer un serveur web dans le sous-réseau public.

Ajoutez :

```
resource "aws_instance" "web" {
  ami                         = data.aws_ami.amazon_linux.id
  instance_type               = "t3.micro"
  subnet_id                   = aws_subnet.public.id
  vpc_security_group_ids      = [aws_security_group.web.id]
  associate_public_ip_address = true

  user_data = <<-EOF
              #!/bin/bash
              dnf install -y nginx
              systemctl enable nginx
              systemctl start nginx
              echo "<h1>Bonjour depuis Terraform</h1>" > /usr/share/nginx/html/index.html
              EOF

  tags = {
    Name = "intro-terraform-web"
  }
}
```

Le script `user_data` est exécuté au démarrage de l'instance. L'installation de Nginx nécessite que l'instance puisse accéder aux dépôts de paquets via Internet.

Exécutez `terraform fmt`, `terraform validate`, puis `terraform plan`. Vérifiez les dépendances et faites valider le plan par le formateur avant `terraform apply`.

> Test : récupérez l'adresse IP publique dans la console AWS. Depuis un navigateur, ouvrez `http://IP_PUBLIQUE`. Le chargement peut prendre quelques minutes après la création de l'instance.

6. Outputs : exposer les résultats

Étape 12 — Déclarer les outputs

Objectif : Rendre accessibles les informations utiles produites par la configuration.

Créez `outputs.tf` et ajoutez :

```
output "vpc_id" {
  description = "Identifiant du VPC"
  value       = aws_vpc.formation.id
}

output "instance_id" {
  description = "Identifiant de l'instance EC2"
  value       = aws_instance.web.id
}

output "public_ip" {
  description = "Adresse IP publique du serveur web"
```

```
  value       = aws_instance.web.public_ip
}

output "website_url" {
  description = "URL du serveur web"
  value       = "http://${aws_instance.web.public_ip}"
}
```

Exécutez `terraform apply` si nécessaire, puis `terraform output`. Vous pouvez également afficher un output précis avec `terraform output public_ip`.

> Les outputs facilitent la consultation des résultats et peuvent aussi transmettre des valeurs à un module parent.

7. Variables : rendre le projet configurable

Étape 13 — Remplacer les valeurs fixes par des variables

Objectif : Éviter de modifier le code pour changer les paramètres du déploiement.

Créez `variables.tf` :

```
variable "aws_region" {
  description = "Région AWS de déploiement"
  type        = string
  default     = "eu-west-1"
}

variable "student_id" {
  description = "Identifiant unique du stagiaire"
  type        = string
}

variable "instance_type" {
  description = "Type de l'instance EC2"
  type        = string
  default     = "t3.micro"
}

variable "vpc_cidr" {
  description = "Plage CIDR du VPC"
  type        = string
  default     = "10.20.0.0/16"
}

variable "subnet_cidr" {
  description = "Plage CIDR du sous-réseau public"
  type        = string
  default     = "10.20.1.0/24"
}
```

Modifiez le provider pour utiliser la variable :

```
provider "aws" {
  region = var.aws_region
}
```

Remplacez dans les ressources les CIDR écrits en dur par `var.vpc_cidr` et `var.subnet_cidr`. Remplacez aussi le type d'instance par `var.instance_type`.

Étape 14 — Fournir les valeurs dans terraform.tfvars

Objectif : Séparer les paramètres du code Terraform.

Créez `terraform.tfvars` :

```
aws_region    = "eu-west-1"
student_id    = "stagiaire01"
instance_type = "t3.micro"
vpc_cidr      = "10.20.0.0/16"
subnet_cidr   = "10.20.1.0/24"
```

Adaptez `student_id` à votre identifiant. Le fichier `terraform.tfvars` est chargé automatiquement par Terraform.

Étape 15 — Ajouter une validation

Objectif : Contrôler la forme de l'identifiant stagiaire avant le déploiement.

Dans la déclaration de `student_id`, ajoutez un bloc `validation` :

```
variable "student_id" {
  description = "Identifiant unique du stagiaire"
  type        = string

  validation {
    condition     = can(regex("^[a-z0-9-]+$", var.student_id))
    error_message = "Utilisez uniquement des minuscules, chiffres et tirets."
  }
}
```

Testez une valeur valide puis une valeur invalide. Observez à quel moment Terraform signale l'erreur.

Ne placez pas de secrets AWS dans le fichier tfvars. Les identifiants AWS doivent être fournis par le mécanisme d'authentification configuré pour le compte de formation.

8. Locals : factoriser les expressions

Étape 16 — Créer des valeurs locales

Objectif : Calculer une valeur commune et l'utiliser à plusieurs endroits.

Créez `locals.tf` :

```
locals {
  name_prefix = "intro-${var.student_id}"
  common_tags = {
    Project   = "formation-terraform"
    Student   = var.student_id
    ManagedBy = "terraform"
  }
}
```

Remplacez les tags `Name` des ressources par des noms basés sur `local.name_prefix`. Ajoutez `local.common_tags` aux tags des ressources qui acceptent les tags.

```
tags = merge(local.common_tags, {
  Name = "${local.name_prefix}-vpc"
})
```

Un local n'est pas une variable d'entrée : sa valeur est calculée à partir de la configuration. Il évite de répéter les mêmes expressions.

Étape 17 — Observer l'impact des changements

Objectif : Comprendre comment une modification de variable ou de local influence le plan.

Modifiez l'identifiant stagiaire dans `terraform.tfvars` et exécutez `terraform plan`.

N'appliquez pas le changement sans examiner les ressources qui seraient remplacées ou modifiées. Remettez ensuite l'identifiant initial.

Un changement de nom ou de tag peut provoquer une mise à jour ou un remplacement selon la ressource et l'attribut concernés. Le plan Terraform est la référence à examiner.

9. Modules : rendre le réseau réutilisable

Vous allez déplacer les ressources réseau dans un module local. Le projet racine continuera à gérer l'instance EC2 et le groupe de sécurité, mais il consommera les identifiants du réseau fournis par le module.

Principe

Un module est un ensemble de fichiers Terraform regroupés dans un répertoire. Le répertoire racine est lui-même le module racine. Un bloc `module` permet d'appeler un autre module et de lui transmettre des variables.

Étape 18 — Créer l'arborescence du module

Objectif : Préparer un module chargé de créer le VPC, le sous-réseau public, l'Internet Gateway et la route.

Créez le répertoire `modules/network` et les fichiers `main.tf`, `variables.tf` et `outputs.tf` à l'intérieur.

Étape 19 — Déclarer les entrées du module

Objectif : Rendre le module indépendant des valeurs propres à un stagiaire.

Dans `modules/network/variables.tf`, déclarez :

```
variable "name_prefix" {
  type        = string
  description = "Préfixe de nommage"
}

variable "vpc_cidr" {
  type        = string
  description = "CIDR du VPC"
}

variable "subnet_cidr" {
  type        = string
  description = "CIDR du sous-réseau public"
}
```

Étape 20 — Déplacer les ressources réseau

Objectif : Faire gérer le réseau par le module.

Dans `modules/network/main.tf`, déplacez les ressources VPC, Internet Gateway, subnet, route table et association de route table créées précédemment.

Remplacez les valeurs fixes par les variables du module et conservez les références entre ressources. Exemple de VPC :

```
resource "aws_vpc" "this" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "${var.name_prefix}-vpc"
  }
}
```

Pour les autres ressources du module, utilisez `aws_vpc.this.id` comme référence au VPC et nommez les ressources avec `var.name_prefix`.

Étape 21 — Exposer les résultats du module

Objectif : Permettre au module racine de récupérer les identifiants nécessaires.

Dans `modules/network/outputs.tf`, ajoutez :

```
output "vpc_id" {
  value = aws_vpc.this.id
}

output "subnet_id" {
  value = aws_subnet.public.id
}
```

Assurez-vous que les noms locaux utilisés dans ces outputs correspondent bien aux ressources que vous avez déplacées dans le module.

Étape 22 — Appeler le module depuis le module racine

Objectif : Transmettre les paramètres et récupérer les outputs du module.

Dans le `main.tf` racine, remplacez les anciennes ressources réseau par :

```
module "network" {
  source      = "./modules/network"
```

```
  name_prefix = local.name_prefix
  vpc_cidr    = var.vpc_cidr
  subnet_cidr = var.subnet_cidr
}
```

Dans les ressources racines restantes, remplacez les références directes aux anciennes ressources réseau par les outputs du module :

```
# Exemple dans le Security Group
vpc_id = module.network.vpc_id

# Exemple dans l'instance EC2
subnet_id = module.network.subnet_id
```

Le groupe de sécurité et l'instance EC2 restent dans le module racine. Ils utilisent désormais les sorties du module réseau.

Étape 23 — Réinitialiser et contrôler la configuration modulaire

Objectif : Faire prendre en compte le nouveau module et examiner le plan.

Exécutez `terraform init`, puis `terraform fmt -recursive`, `terraform validate` et `terraform plan`.

Si le projet avait déjà été appliqué avant le déplacement, ne lancez pas `apply` tant que le plan ne montre pas clairement que les ressources existantes sont conservées.

> Point de vigilance important : déplacer des blocs de ressources vers un module change leur adresse dans l'état Terraform. Terraform peut alors proposer de détruire les anciennes adresses et de créer de nouvelles ressources. Pour une migration d'état, il faut traiter explicitement ce changement d'adresse, par exemple avec des blocs `moved` adaptés, ou une opération d'état maîtrisée. Pour ce TP, le plus simple est de faire le refactoring avant le premier `apply`, ou de repartir d'un environnement détruit et d'un état propre.

10. Vérification finale et nettoyage

Étape 24 — Vérifier les outputs et le site

Objectif : Confirmer que le déploiement final fonctionne.

Exécutez `terraform output`.

Ouvrez l'URL affichée dans le navigateur.

Vérifiez dans la console AWS que l'instance est dans le VPC et le sous-réseau attendus.

Étape 25 — Détruire les ressources

Objectif : Supprimer les ressources créées pendant le TP.

Exécutez `terraform destroy`, lisez attentivement le plan et confirmez avec `yes`.

Vérifiez dans la console AWS que l'instance EC2, le groupe de sécurité, le réseau et les ressources associées ont bien été supprimés.

> Ne supprimez pas le fichier `terraform.tfstate` avant d'avoir correctement détruit les ressources. L'état permet à Terraform de savoir quelles ressources il gère.

11. Questions de compréhension

- Quelle est la différence entre un provider et une ressource ?
- Quelle est la différence entre une ressource et une data source ?
- À quoi sert `terraform init` ?
- Quelle différence faites-vous entre `terraform plan` et `terraform apply` ?
- Pourquoi la référence `aws_vpc.formation.id` crée-t-elle une dépendance ?

- À quoi sert un output ?
- Quelle différence faites-vous entre une variable et un local ?
- Pourquoi transmettre des variables à un module plutôt que coder les valeurs directement dans le module ?
- Comment un module racine récupère-t-il une valeur exposée par un module enfant ?
- Pourquoi faut-il examiner le plan après avoir déplacé des ressources dans un module ?

12. Défis facultatifs

- Ajoutez un output affichant le CIDR du VPC.
- Ajoutez une variable pour le nom du projet et utilisez-la dans le local `name_prefix`.
- Ajoutez une validation sur le CIDR du VPC ou du sous-réseau.
- Ajoutez un second environnement logique en appelant le module réseau avec des valeurs différentes dans un répertoire de travail séparé. Ne déployez pas les deux environnements sans validation du formateur.
- Exécutez `terraform graph` et identifiez les dépendances entre les principales ressources.

13. Livrables attendus

- Le répertoire du projet avec les fichiers Terraform organisés.
- Le module local `modules/network` avec ses entrées et ses outputs.
- Les outputs affichant les identifiants utiles et l'URL du site.
- Une capture ou une démonstration du site Nginx accessible en HTTP.
- Les réponses aux questions de compréhension.
- La confirmation que les ressources ont été détruites à la fin du TP.

14. Aide-mémoire des commandes

```
terraform version
terraform init
terraform providers
terraform fmt
terraform fmt -recursive
terraform validate
terraform plan
terraform apply
terraform output
terraform output public_ip
terraform graph
terraform destroy
```

Les commandes `terraform plan`, `apply` et `destroy` doivent être utilisées en lisant systématiquement les changements proposés. Ne confirmez jamais un plan que vous ne comprenez pas.

Annexe — Points de contrôle formateur

- La région AWS est autorisée et identique entre le provider et l'AMI recherchée.
- Chaque stagiaire dispose d'un identifiant distinct.
- Les stagiaires n'ont pas ajouté de règle SSH publique inutile.

- Le sous-réseau possède une route par défaut vers l'Internet Gateway.
- Le Security Group autorise HTTP sur le port 80.
- L'AMI sélectionnée est disponible dans la région et l'architecture attendues.
- Le plan final après modularisation ne prévoit pas de remplacement inattendu.
- Chaque stagiaire a exécuté terraform destroy et vérifié le nettoyage.

Ce TP est un support pédagogique. Les choix de réseau et de sécurité sont simplifiés pour l'exercice et ne constituent pas une architecture de production.